Practica 4

1. Repetir la verificación del circuito sumador de 4 bits + 4 bits que describiste en la práctica utilizando los contenidos de testbench automatizado con assertions para verificar automáticamente todos los casos de entrada.

```
//Full_Adder_4b TestBentch
`timescale 1ns/1ns
`include "Full_Adder_4b.v" //Import from wich module we are doig the testbench

module Full_Adder_4b_tb();

integer i; //Varaiable for counting
integer A_var,B_var; //Variables for input combinations
reg [4:0] result; //Variable to store expected result
reg  [3:0]A,B;  //Create registers for our inputs
wire [4:0]S;   //Create wires for our outputs

Full_Adder_4b uut (A,B,S); //Call our module from Full_Adder_M.v

initial begin
    $dumpfile("Full_Adder_4b_tb.vcd"); //Create the file that will give us the waveform of our output
    $dumpvars(0,Full_Adder_4b_tb);

    //Test 10 random cases for the Adder

    for (A_var = 0; A_var < 16; A_var = A_var + 1)begin //Counter for all values for A
        for (B_var = 0; B_var < 16; B_var = B_var + 1)begin //Counter for all values for B
            result = A_var + B_var; //Expected result
            A = A_var; //Assign value to A
            B = B_var; //Assign value to B
            #10; //Wait 10 time units

            if (S !== result) begin //Compare expected result with output from the module
                $display("Test failed for A = %d, B = %d ---> Expected S = %d, got S = %d", A, B, result, S);
            end else begin
                $display("Test passed for A = %d, B = %d ---> S = %d", A, B, S);
            end
        end
    end

    $display("Test completed"); //Show message in console when test is completed
    $finish; //End the simulation

end

endmodule
```

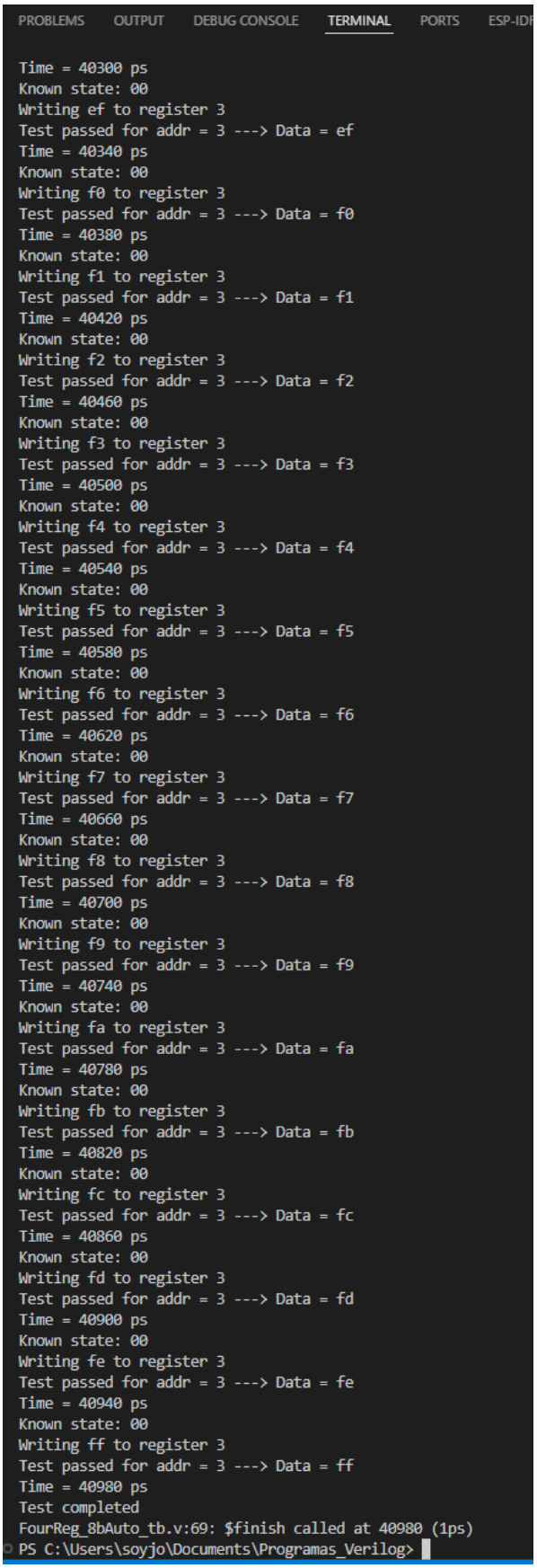
```
>>
VCD info: dumpfile Full_Adder_4b_tb.vcd opened for output.
Test passed for A = 11, B = 12 ---> S = 23
Test passed for A = 11, B = 13 ---> S = 24
Test passed for A = 11, B = 14 ---> S = 25
Test passed for A = 11, B = 15 ---> S = 26
Test passed for A = 12, B =  0 ---> S = 12
Test passed for A = 12, B =  1 ---> S = 13
Test passed for A = 12, B =  2 ---> S = 14
Test passed for A = 12, B =  3 ---> S = 15
Test passed for A = 12, B =  4 ---> S = 16
Test passed for A = 12, B =  5 ---> S = 17
Test passed for A = 12, B =  6 ---> S = 18
Test passed for A = 12, B =  7 ---> S = 19
Test passed for A = 12, B =  8 ---> S = 20
Test passed for A = 12, B =  9 ---> S = 21
Test passed for A = 12, B = 10 ---> S = 22
Test passed for A = 12, B = 11 ---> S = 23
Test passed for A = 12, B = 12 ---> S = 24
Test passed for A = 12, B = 13 ---> S = 25
Test passed for A = 12, B = 14 ---> S = 26
Test passed for A = 12, B = 15 ---> S = 27
Test passed for A = 13, B =  0 ---> S = 13
Test passed for A = 13, B =  1 ---> S = 14
Test passed for A = 13, B =  2 ---> S = 15
Test passed for A = 13, B =  3 ---> S = 16
Test passed for A = 13, B =  4 ---> S = 17
Test passed for A = 13, B =  5 ---> S = 18
Test passed for A = 13, B =  6 ---> S = 19
Test passed for A = 13, B =  7 ---> S = 20
Test passed for A = 13, B =  8 ---> S = 21
Test passed for A = 13, B =  9 ---> S = 22
Test passed for A = 13, B = 10 ---> S = 23
Test passed for A = 13, B = 11 ---> S = 24
Test passed for A = 13, B = 12 ---> S = 25
Test passed for A = 13, B = 13 ---> S = 26
Test passed for A = 13, B = 14 ---> S = 27
Test passed for A = 13, B = 15 ---> S = 28
Test passed for A = 14, B =  0 ---> S = 14
Test passed for A = 14, B =  1 ---> S = 15
Test passed for A = 14, B =  2 ---> S = 16
Test passed for A = 14, B =  3 ---> S = 17
Test passed for A = 14, B =  4 ---> S = 18
Test passed for A = 14, B =  5 ---> S = 19
Test passed for A = 14, B =  6 ---> S = 20
Test passed for A = 14, B =  7 ---> S = 21
Test passed for A = 14, B =  8 ---> S = 22
Test passed for A = 14, B =  9 ---> S = 23
Test passed for A = 14, B = 10 ---> S = 24
Test passed for A = 14, B = 11 ---> S = 25
Test passed for A = 14, B = 12 ---> S = 26
Test passed for A = 14, B = 13 ---> S = 27
Test passed for A = 14, B = 14 ---> S = 28
Test passed for A = 14, B = 15 ---> S = 29
Test passed for A = 15, B =  0 ---> S = 15
Test passed for A = 15, B =  1 ---> S = 16
Test passed for A = 15, B =  2 ---> S = 17
Test passed for A = 15, B =  3 ---> S = 18
Test passed for A = 15, B =  4 ---> S = 19
Test passed for A = 15, B =  5 ---> S = 20
Test passed for A = 15, B =  6 ---> S = 21
Test passed for A = 15, B =  7 ---> S = 22
Test passed for A = 15, B =  8 ---> S = 23
Test passed for A = 15, B =  9 ---> S = 24
Test passed for A = 15, B = 10 ---> S = 25
Test passed for A = 15, B = 11 ---> S = 26
Test passed for A = 15, B = 12 ---> S = 27
Test passed for A = 15, B = 13 ---> S = 28
Test passed for A = 15, B = 14 ---> S = 29
Test passed for A = 15, B = 15 ---> S = 30
Test completed
Full_Adder_4b_tb.v:37: $finish called at 2560 (1ns)
```

```
//Full_Adder_4b TestBentch
`timescale 1ns/1ns
`include "Full_Adder_4b.v" //Import from wich module we are doig the testbench

module Full_Adder_4b_tb();

integer i; //Varaiable for counting
integer A_var,B_var; //Variables for input combinations
reg [4:0] result; //Variable to store expected result
reg  [3:0]A,B;  //Create registers for our inputs
wire [4:0]S;   //Create wires for our outputs

Full_Adder_4b uut (A,B,S); //Call our module from Full_Adder_M.v

initial begin
  $dumpfile("Full_Adder_4b_tb.vcd"); //Create the file that will give us the waveform of our
output
  $dumpvars(0,Full_Adder_4b_tb);

  //Test 10 random cases for the Adder

  for (A_var = 0; A_var < 16; A_var = A_var + 1)begin //Counter for all values for A
    for (B_var = 0; B_var < 16; B_var = B_var + 1)begin //Counter for all values for B
      result = A_var + B_var; //Expected result
      A = A_var; //Assign value to A
      B = B_var; //Assign value to B
      #10; //Wait 10 time units

      if (S !== result) begin //Compare expected result with output from the module
        $display("Test failed for A = %d, B = %d ---> Expected S = %d, got S = %d", A, B, result,
S);
      end else begin
        $display("Test passed for A = %d, B = %d ---> S = %d", A, B, S);
      end
    end
  end

  $display("Test completed"); //Show message in console when test is completed
  $finish; //End the simulation

end

endmodule
```

2. Repetir la verificación del circuito del banco de registros que describiste en la práctica 3, utilizando los contenidos de testbench automatizado con assertions para verificar automáticamente todos los casos de entrada para algún caso de estado actual.

```
//Four registers with 8-bit width, write enable and reset testbench
`timescale 1ps/1ps
`include "FourReg_8b.v"

module FourReg_8bAuto_tb ();
integer addr_var, write_data_var;
integer response_char;
reg [7:0] result;
reg clk, rst_n, en_write;
reg [1:0] addr;
reg [7:0] write_data;
wire [7:0] out_0, out_1, out_2, out_3;

FourReg_8b uut (clk, rst_n, en_write, addr, write_data, out_0, out_1, out_2, out_3);

initial begin
    $dumpfile("FourReg_8bAuto_tb.vcd");
    $dumpvars(0,FourReg_8bAuto_tb);

    rst_n = 0; en_write = 0;
    clk = 0;
    #10
    clk = 1;
    rst_n = 1; en_write = 1;
    #10;

    for (addr_var = 0; addr_var < 4; addr_var = addr_var + 1) begin
        addr = addr_var[1:0];
        for (write_data_var = 0; write_data_var < 256; write_data_var = write_data_var + 1) begin

            write_data = 8'h00;
            clk = 0; #10;
            clk = 1; #10;

            if (addr == 2'b00) begin
                $display("Known state: %h", out_0);
            end else if (addr == 2'b01) begin
                $display("Known state: %h", out_1);
            end else if (addr == 2'b10) begin
                $display("Known state: %h", out_2);
            end else begin
                $display("Known state: %h", out_3);
            end

            write_data = write_data_var[7:0];
            clk = 0; #10;
            $display("Writing %h to register %d", write_data, addr);
            clk = 1; #10;

            case (addr)
                2'b00: result = out_0;
                2'b01: result = out_1;
                2'b10: result = out_2;
                2'b11: result = out_3;
            endcase
```

```
56
57              if (result != write_data) begin
58                  $display("Test failed for addr = %d ---> Expected data = %h, got data = %h", addr, write_data, result);
59                  $display("Time = %0t ps", $time);
60              end else begin
61                  $display("Test passed for addr = %d ---> Data = %h", addr, write_data);
62                  $display("Time = %0t ps", $time);
63              end
64
65          end
66      end
67
68      $display("Test completed");
69      $finish;
70  end
71  endmodule
```

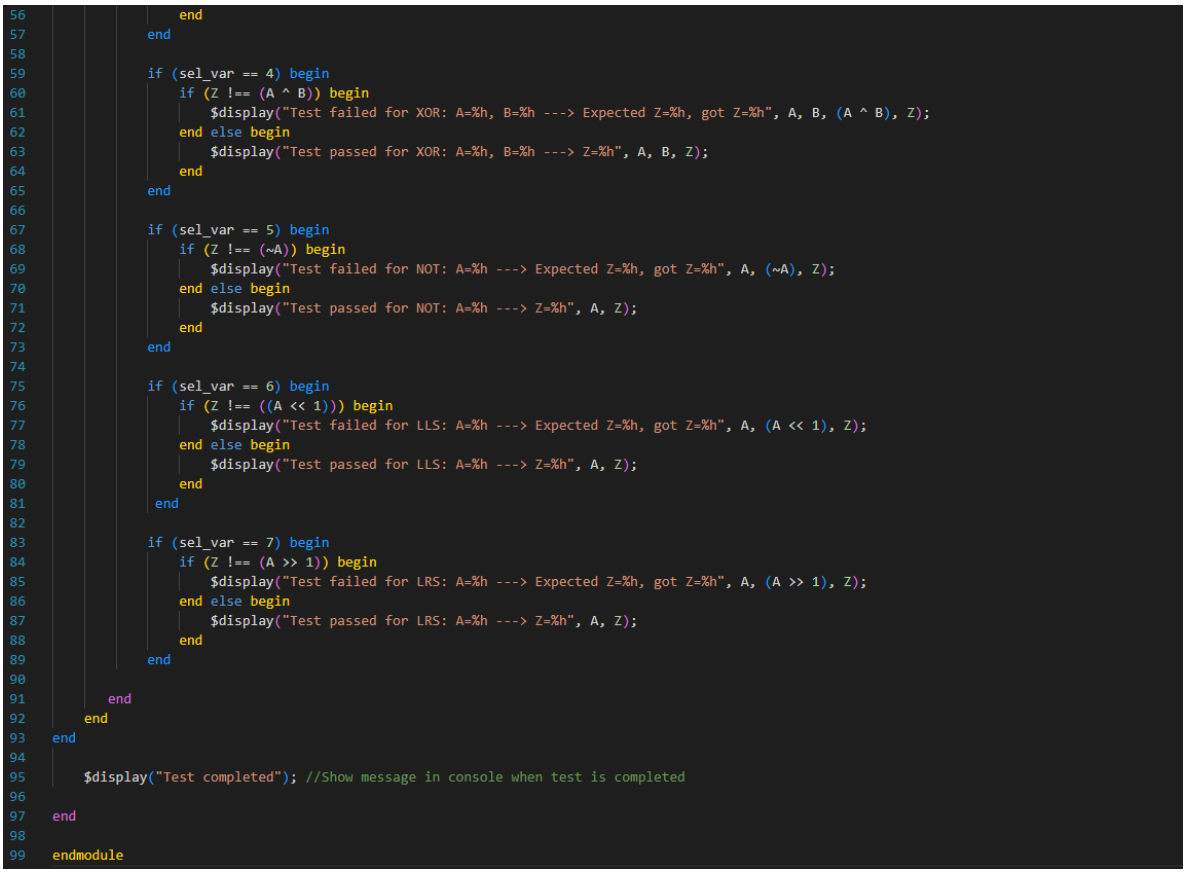

PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
ESP-IDF
Time = 40300 ps
Known state: 00
Writing ef to register 3
Test passed for addr = 3 ---> Data = ef
Time = 40340 ps
Known state: 00
Writing f0 to register 3
Test passed for addr = 3 ---> Data = f0
Time = 40380 ps
Known state: 00
Writing f1 to register 3
Test passed for addr = 3 ---> Data = f1
Time = 40420 ps
Known state: 00
Writing f2 to register 3
Test passed for addr = 3 ---> Data = f2
Time = 40460 ps
Known state: 00
Writing f3 to register 3
Test passed for addr = 3 ---> Data = f3
Time = 40500 ps
Known state: 00
Writing f4 to register 3
Test passed for addr = 3 ---> Data = f4
Time = 40540 ps
Known state: 00
Writing f5 to register 3
Test passed for addr = 3 ---> Data = f5
Time = 40580 ps
Known state: 00
Writing f6 to register 3
Test passed for addr = 3 ---> Data = f6
Time = 40620 ps
Known state: 00
Writing f7 to register 3
Test passed for addr = 3 ---> Data = f7
Time = 40660 ps
Known state: 00
Writing f8 to register 3
Test passed for addr = 3 ---> Data = f8
Time = 40700 ps
Known state: 00
Writing f9 to register 3
Test passed for addr = 3 ---> Data = f9
Time = 40740 ps
Known state: 00
Writing fa to register 3
Test passed for addr = 3 ---> Data = fa
Time = 40780 ps
Known state: 00
Writing fb to register 3
Test passed for addr = 3 ---> Data = fb
Time = 40820 ps
Known state: 00
Writing fc to register 3
Test passed for addr = 3 ---> Data = fc
Time = 40860 ps
Known state: 00
Writing fd to register 3
Test passed for addr = 3 ---> Data = fd
Time = 40900 ps
Known state: 00
Writing fe to register 3
Test passed for addr = 3 ---> Data = fe
Time = 40940 ps
Known state: 00
Writing ff to register 3
Test passed for addr = 3 ---> Data = ff
Time = 40980 ps
Test completed
FourReg_8bAuto_tb.v:69: $finish called at 40980 (1ps)
PS C:\Users\soyjo\Documents\Programas_Verilog>

```
//Four registers with 8-bit width, write enable and reset testbench
`timescale 1ps/1ps
`include "FourReg_8b.v"

module FourReg_8bAuto_tb ();
integer addr_var, write_data_var;
integer response_char;
reg [7:0] result;
reg clk, rst_n, en_write;
reg [1:0] addr;
reg [7:0] write_data;
wire [7:0] out_0, out_1, out_2, out_3;

FourReg_8b uut (clk, rst_n, en_write, addr, write_data, out_0, out_1, out_2, out_3);

initial begin
  $dumpfile("FourReg_8bAuto_tb.vcd");
  $dumpvars(0,FourReg_8bAuto_tb);

  rst_n = 0; en_write = 0;
  clk = 0;
  #10
  clk = 1;
  rst_n = 1; en_write = 1;
  #10;

  for (addr_var = 0; addr_var < 4; addr_var = addr_var + 1) begin
    addr = addr_var[1:0];
    for (write_data_var = 0; write_data_var < 256; write_data_var = write_data_var + 1) begin

      write_data = 8'h00;
      clk = 0; #10;
      clk = 1; #10;

      if (addr == 2'b00) begin
        $display("Known state: %h", out_0);
      end else if (addr == 2'b01) begin
        $display("Known state: %h", out_1);
      end else if (addr == 2'b10) begin
        $display("Known state: %h", out_2);
      end else begin
        $display("Known state: %h", out_3);
      end
```

```
        write_data = write_data_var[7:0];
        clk = 0; #10;
        $display("Writing %h to register %d", write_data, addr);
        clk = 1; #10;

        case (addr)
          2'b00: result = out_0;
          2'b01: result = out_1;
          2'b10: result = out_2;
          2'b11: result = out_3;
        endcase

        if (result != write_data) begin
          $display("Test failed for addr = %d ---> Expected data = %h, got data = %h", addr,
write_data, result);
          $display("Time = %0t ps", $time);
        end else begin
          $display("Test passed for addr = %d ---> Data = %h", addr, write_data);
          $display("Time = %0t ps", $time);
        end

      end
    end

    $display("Test completed");
    $finish;
end
endmodule
```

3. Repetir la verificación del circuito ALU que describiste en la práctica 2, utilizando los contenidos de testbench automatizado con assertions para verificar automáticamente todos los casos de entrada.

```
//ALU_M testbench
`timescale 1ns/1ps
`include "ALU_M.v" //Import the file to which we are making its testbench

module ALU_MAuto_tb ();
integer sel_var, A_var, B_var; //Variable for counting
reg  [7:0] A,B; //8-bit inputs
reg  [2:0] sel; //3-bit select input
wire [7:0] Z; //8-bit output
wire zero,carry,sign; //bitFlags

//Instantiate the module under test
ALU_M uut (A,B,sel,Z,zero,carry,sign);

initial begin

    $dumpfile("ALU_MAuto_tb.vcd"); //Create the file that will give us the waveform of our output
    $dumpvars(0,ALU_MAuto_tb);     //Record all variable changes in the testbench

for (sel_var = 0; sel_var < 9; sel_var = sel_var + 1) begin
    sel = sel_var[2:0];
  for (A_var = 0; A_var < 256; A_var = A_var + 1) begin
      A = A_var[7:0];
      for (B_var = 0; B_var < 256; B_var = B_var + 1) begin
          B = B_var[7:0];
           #10;
           if (sel_var == 0) begin
               if (Z !== (A + B)) begin
                   $display("Test failed for ADD: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A + B), Z);
               end else begin
                   $display("Test passed for ADD: A=%h, B=%h ---> Z=%h", A, B, Z);
               end
           end

           if (sel_var == 1) begin
               if (Z !== (A - B)) begin
                   $display("Test failed for SUB: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A - B), Z);
               end else begin
                   $display("Test passed for SUB: A=%h, B=%h ---> Z=%h", A, B, Z);
               end
           end

           if (sel_var == 2) begin
               if (Z !== (A & B)) begin
                   $display("Test failed for AND: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A & B), Z);
               end else begin
                   $display("Test passed for AND: A=%h, B=%h ---> Z=%h", A, B, Z);
               end
           end

           if (sel_var == 3) begin
               if (Z !== (A | B)) begin
                   $display("Test failed for OR: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A | B), Z);
               end else begin
                   $display("Test passed for OR: A=%h, B=%h ---> Z=%h", A, B, Z);
               end
           end
```



```
                end
            end

            if (sel_var == 4) begin
                if (Z !== (A ^ B)) begin
                    $display("Test failed for XOR: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A ^ B), Z);
                end else begin
                    $display("Test passed for XOR: A=%h, B=%h ---> Z=%h", A, B, Z);
                end
            end

            if (sel_var == 5) begin
                if (Z !== (~A)) begin
                    $display("Test failed for NOT: A=%h ---> Expected Z=%h, got Z=%h", A, (~A), Z);
                end else begin
                    $display("Test passed for NOT: A=%h ---> Z=%h", A, Z);
                end
            end

            if (sel_var == 6) begin
                if (Z !== ((A << 1))) begin
                    $display("Test failed for LLS: A=%h ---> Expected Z=%h, got Z=%h", A, (A << 1), Z);
                end else begin
                    $display("Test passed for LLS: A=%h ---> Z=%h", A, Z);
                end
             end

            if (sel_var == 7) begin
                if (Z !== (A >> 1)) begin
                    $display("Test failed for LRS: A=%h ---> Expected Z=%h, got Z=%h", A, (A >> 1), Z);
                end else begin
                    $display("Test passed for LRS: A=%h ---> Z=%h", A, Z);
                end
            end

        end
    end
end

    $display("Test completed"); //Show message in console when test is completed

end

endmodule
```

Algunos ejemplos, la simulación duro alrededor de 5 minutos.

```
Test passed for ADD: A=4c, B=9a ---> Z=e6
Test passed for ADD: A=4c, B=9b ---> Z=e7
Test passed for ADD: A=4c, B=9c ---> Z=e8
Test passed for ADD: A=4c, B=9d ---> Z=e9
Test passed for ADD: A=4c, B=9e ---> Z=ea
Test passed for ADD: A=4c, B=9f ---> Z=eb
Test passed for ADD: A=4c, B=a0 ---> Z=ec
Test passed for ADD: A=4c, B=a1 ---> Z=ed
Test passed for ADD: A=4c, B=a2 ---> Z=ee
Test passed for ADD: A=4c, B=a3 ---> Z=ef
Test passed for ADD: A=4c, B=a4 ---> Z=f0
Test passed for ADD: A=4c, B=a5 ---> Z=f1
Test passed for ADD: A=4c, B=a6 ---> Z=f2
Test passed for ADD: A=4c, B=a7 ---> Z=f3
Test passed for ADD: A=4c, B=a8 ---> Z=f4
Test passed for ADD: A=4c, B=a9 ---> Z=f5
Test passed for ADD: A=4c, B=aa ---> Z=f6
Test passed for ADD: A=4c, B=ab ---> Z=f7
Test passed for ADD: A=4c, B=ac ---> Z=f8
Test passed for ADD: A=4c, B=ad ---> Z=f9
Test passed for ADD: A=4c, B=ae ---> Z=fa
Test passed for ADD: A=4c, B=af ---> Z=fb
Test passed for ADD: A=4c, B=b0 ---> Z=fc
Test passed for ADD: A=4c, B=b1 ---> Z=fd
Test passed for ADD: A=4c, B=b2 ---> Z=fe
Test passed for ADD: A=4c, B=b3 ---> Z=ff
Test passed for ADD: A=4c, B=b4 ---> Z=00
Test passed for ADD: A=4c, B=b5 ---> Z=01
Test passed for ADD: A=4c, B=b6 ---> Z=02
Test passed for ADD: A=4c, B=b7 ---> Z=03
Test passed for ADD: A=4c, B=b8 ---> Z=04
Test passed for ADD: A=4c, B=b9 ---> Z=05
Test passed for ADD: A=4c, B=ba ---> Z=06
Test passed for ADD: A=4c, B=bb ---> Z=07
Test passed for ADD: A=4c, B=bc ---> Z=08
Test passed for ADD: A=4c, B=bd ---> Z=09
Test passed for ADD: A=4c, B=be ---> Z=0a
Test passed for ADD: A=4c, B=bf ---> Z=0b
Test passed for ADD: A=4c, B=c0 ---> Z=0c
Test passed for ADD: A=4c, B=c1 ---> Z=0d
Test passed for ADD: A=4c, B=c2 ---> Z=0e
Test passed for ADD: A=4c, B=c3 ---> Z=0f
Test passed for ADD: A=4c, B=c4 ---> Z=10
Test passed for ADD: A=4c, B=c5 ---> Z=11
Test passed for ADD: A=4c, B=c6 ---> Z=12
Test passed for ADD: A=4c, B=c7 ---> Z=13
Test passed for ADD: A=4c, B=c8 ---> Z=14
Test passed for ADD: A=4c, B=c9 ---> Z=15
Test passed for ADD: A=4c, B=ca ---> Z=16
Test passed for ADD: A=4c, B=cb ---> Z=17
Test passed for ADD: A=4c, B=cc ---> Z=18
Test passed for ADD: A=4c, B=cd ---> Z=19
Test passed for ADD: A=4c, B=ce ---> Z=1a
Test passed for ADD: A=4c, B=cf ---> Z=1b
Test passed for ADD: A=4c, B=d0 ---> Z=1c
Test passed for ADD: A=4c, B=d1 ---> Z=1d
Test passed for ADD: A=4c, B=d2 ---> Z=1e
Test passed for ADD: A=4c, B=d3 ---> Z=1f
Test passed for ADD: A=4c, B=d4 ---> Z=20
Test passed for ADD: A=4c, B=d5 ---> Z=21
Test passed for ADD: A=4c, B=d6 ---> Z=22
Test passed for ADD: A=4c, B=d7 ---> Z=23
Test passed for ADD: A=4c, B=d8 ---> Z=24
Test passed for ADD: A=4c, B=d9 ---> Z=25
Test passed for ADD: A=4c, B=da ---> Z=26
Test passed for ADD: A=4c, B=db ---> Z=27
Test passed for ADD: A=4c, B=dc ---> Z=28
```

```
Test passed for SUB: A=fe, B=75 ---> Z=89
Test passed for SUB: A=fe, B=76 ---> Z=88
Test passed for SUB: A=fe, B=77 ---> Z=87
Test passed for SUB: A=fe, B=78 ---> Z=86
Test passed for SUB: A=fe, B=79 ---> Z=85
Test passed for SUB: A=fe, B=7a ---> Z=84
Test passed for SUB: A=fe, B=7b ---> Z=83
Test passed for SUB: A=fe, B=7c ---> Z=82
Test passed for SUB: A=fe, B=7d ---> Z=81
Test passed for SUB: A=fe, B=7e ---> Z=80
Test passed for SUB: A=fe, B=7f ---> Z=7f
Test passed for SUB: A=fe, B=80 ---> Z=7e
Test passed for SUB: A=fe, B=81 ---> Z=7d
Test passed for SUB: A=fe, B=82 ---> Z=7c
Test passed for SUB: A=fe, B=83 ---> Z=7b
Test passed for SUB: A=fe, B=84 ---> Z=7a
Test passed for SUB: A=fe, B=85 ---> Z=79
Test passed for SUB: A=fe, B=86 ---> Z=78
Test passed for SUB: A=fe, B=87 ---> Z=77
Test passed for SUB: A=fe, B=88 ---> Z=76
Test passed for SUB: A=fe, B=89 ---> Z=75
Test passed for SUB: A=fe, B=8a ---> Z=74
Test passed for SUB: A=fe, B=8b ---> Z=73
Test passed for SUB: A=fe, B=8c ---> Z=72
Test passed for SUB: A=fe, B=8d ---> Z=71
Test passed for SUB: A=fe, B=8e ---> Z=70
Test passed for SUB: A=fe, B=8f ---> Z=6f
Test passed for SUB: A=fe, B=90 ---> Z=6e
Test passed for SUB: A=fe, B=91 ---> Z=6d
Test passed for SUB: A=fe, B=92 ---> Z=6c
Test passed for SUB: A=fe, B=93 ---> Z=6b
Test passed for SUB: A=fe, B=94 ---> Z=6a
Test passed for SUB: A=fe, B=95 ---> Z=69
Test passed for SUB: A=fe, B=96 ---> Z=68
Test passed for SUB: A=fe, B=97 ---> Z=67
Test passed for SUB: A=fe, B=98 ---> Z=66
Test passed for SUB: A=fe, B=99 ---> Z=65
Test passed for SUB: A=fe, B=9a ---> Z=64
Test passed for SUB: A=fe, B=9b ---> Z=63
Test passed for SUB: A=fe, B=9c ---> Z=62
Test passed for SUB: A=fe, B=9d ---> Z=61
Test passed for SUB: A=fe, B=9e ---> Z=60
Test passed for SUB: A=fe, B=9f ---> Z=5f
Test passed for SUB: A=fe, B=a0 ---> Z=5e
Test passed for SUB: A=fe, B=a1 ---> Z=5d
Test passed for SUB: A=fe, B=a2 ---> Z=5c
Test passed for SUB: A=fe, B=a3 ---> Z=5b
Test passed for SUB: A=fe, B=a4 ---> Z=5a
Test passed for SUB: A=fe, B=a5 ---> Z=59
Test passed for SUB: A=fe, B=a6 ---> Z=58
Test passed for SUB: A=fe, B=a7 ---> Z=57
Test passed for SUB: A=fe, B=a8 ---> Z=56
Test passed for SUB: A=fe, B=a9 ---> Z=55
Test passed for SUB: A=fe, B=aa ---> Z=54
Test passed for SUB: A=fe, B=ab ---> Z=53
Test passed for SUB: A=fe, B=ac ---> Z=52
Test passed for SUB: A=fe, B=ad ---> Z=51
Test passed for SUB: A=fe, B=ae ---> Z=50
Test passed for SUB: A=fe, B=af ---> Z=4f
Test passed for SUB: A=fe, B=b0 ---> Z=4e
Test passed for SUB: A=fe, B=b1 ---> Z=4d
Test passed for SUB: A=fe, B=b2 ---> Z=4c
Test passed for SUB: A=fe, B=b3 ---> Z=4b
Test passed for SUB: A=fe, B=b4 ---> Z=4a
Test passed for SUB: A=fe, B=b5 ---> Z=49
Test passed for SUB: A=fe, B=b6 ---> Z=48
Test passed for SUB: A=fe, B=b7 ---> Z=47
```

```
Test passed for OR: A=08, B=85 ---> Z=8d
Test passed for OR: A=08, B=86 ---> Z=8e
Test passed for OR: A=08, B=87 ---> Z=8f
Test passed for OR: A=08, B=88 ---> Z=88
Test passed for OR: A=08, B=89 ---> Z=89
Test passed for OR: A=08, B=8a ---> Z=8a
Test passed for OR: A=08, B=8b ---> Z=8b
Test passed for OR: A=08, B=8c ---> Z=8c
Test passed for OR: A=08, B=8d ---> Z=8d
Test passed for OR: A=08, B=8e ---> Z=8e
Test passed for OR: A=08, B=8f ---> Z=8f
Test passed for OR: A=08, B=90 ---> Z=98
Test passed for OR: A=08, B=91 ---> Z=99
Test passed for OR: A=08, B=92 ---> Z=9a
Test passed for OR: A=08, B=93 ---> Z=9b
Test passed for OR: A=08, B=94 ---> Z=9c
Test passed for OR: A=08, B=95 ---> Z=9d
Test passed for OR: A=08, B=96 ---> Z=9e
Test passed for OR: A=08, B=97 ---> Z=9f
Test passed for OR: A=08, B=98 ---> Z=98
Test passed for OR: A=08, B=99 ---> Z=99
Test passed for OR: A=08, B=9a ---> Z=9a
Test passed for OR: A=08, B=9b ---> Z=9b
Test passed for OR: A=08, B=9c ---> Z=9c
Test passed for OR: A=08, B=9d ---> Z=9d
Test passed for OR: A=08, B=9e ---> Z=9e
Test passed for OR: A=08, B=9f ---> Z=9f
Test passed for OR: A=08, B=a0 ---> Z=a8
Test passed for OR: A=08, B=a1 ---> Z=a9
Test passed for OR: A=08, B=a2 ---> Z=aa
Test passed for OR: A=08, B=a3 ---> Z=ab
Test passed for OR: A=08, B=a4 ---> Z=ac
Test passed for OR: A=08, B=a5 ---> Z=ad
Test passed for OR: A=08, B=a6 ---> Z=ae
Test passed for OR: A=08, B=a7 ---> Z=af
Test passed for OR: A=08, B=a8 ---> Z=a8
Test passed for OR: A=08, B=a9 ---> Z=a9
Test passed for OR: A=08, B=aa ---> Z=aa
Test passed for OR: A=08, B=ab ---> Z=ab
Test passed for OR: A=08, B=ac ---> Z=ac
Test passed for OR: A=08, B=ad ---> Z=ad
Test passed for OR: A=08, B=ae ---> Z=ae
Test passed for OR: A=08, B=af ---> Z=af
Test passed for OR: A=08, B=b0 ---> Z=b8
Test passed for OR: A=08, B=b1 ---> Z=b9
Test passed for OR: A=08, B=b2 ---> Z=ba
Test passed for OR: A=08, B=b3 ---> Z=bb
Test passed for OR: A=08, B=b4 ---> Z=bc
Test passed for OR: A=08, B=b5 ---> Z=bd
Test passed for OR: A=08, B=b6 ---> Z=be
Test passed for OR: A=08, B=b7 ---> Z=bf
Test passed for OR: A=08, B=b8 ---> Z=b8
Test passed for OR: A=08, B=b9 ---> Z=b9
Test passed for OR: A=08, B=ba ---> Z=ba
Test passed for OR: A=08, B=bb ---> Z=bb
Test passed for OR: A=08, B=bc ---> Z=bc
Test passed for OR: A=08, B=bd ---> Z=bd
Test passed for OR: A=08, B=be ---> Z=be
Test passed for OR: A=08, B=bf ---> Z=bf
Test passed for OR: A=08, B=c0 ---> Z=c8
Test passed for OR: A=08, B=c1 ---> Z=c9
Test passed for OR: A=08, B=c2 ---> Z=ca
Test passed for OR: A=08, B=c3 ---> Z=cb
Test passed for OR: A=08, B=c4 ---> Z=cc
Test passed for OR: A=08, B=c5 ---> Z=cd
Test passed for OR: A=08, B=c6 ---> Z=ce
Test passed for OR: A=08, B=c7 ---> Z=cf
```

```
Test passed for XOR: A=14, B=4c ---> Z=58
Test passed for XOR: A=14, B=4d ---> Z=59
Test passed for XOR: A=14, B=4e ---> Z=5a
Test passed for XOR: A=14, B=4f ---> Z=5b
Test passed for XOR: A=14, B=50 ---> Z=44
Test passed for XOR: A=14, B=51 ---> Z=45
Test passed for XOR: A=14, B=52 ---> Z=46
Test passed for XOR: A=14, B=53 ---> Z=47
Test passed for XOR: A=14, B=54 ---> Z=40
Test passed for XOR: A=14, B=55 ---> Z=41
Test passed for XOR: A=14, B=56 ---> Z=42
Test passed for XOR: A=14, B=57 ---> Z=43
Test passed for XOR: A=14, B=58 ---> Z=4c
Test passed for XOR: A=14, B=59 ---> Z=4d
Test passed for XOR: A=14, B=5a ---> Z=4e
Test passed for XOR: A=14, B=5b ---> Z=4f
Test passed for XOR: A=14, B=5c ---> Z=48
Test passed for XOR: A=14, B=5d ---> Z=49
Test passed for XOR: A=14, B=5e ---> Z=4a
Test passed for XOR: A=14, B=5f ---> Z=4b
Test passed for XOR: A=14, B=60 ---> Z=74
Test passed for XOR: A=14, B=61 ---> Z=75
Test passed for XOR: A=14, B=62 ---> Z=76
Test passed for XOR: A=14, B=63 ---> Z=77
Test passed for XOR: A=14, B=64 ---> Z=70
Test passed for XOR: A=14, B=65 ---> Z=71
Test passed for XOR: A=14, B=66 ---> Z=72
Test passed for XOR: A=14, B=67 ---> Z=73
Test passed for XOR: A=14, B=68 ---> Z=7c
Test passed for XOR: A=14, B=69 ---> Z=7d
Test passed for XOR: A=14, B=6a ---> Z=7e
Test passed for XOR: A=14, B=6b ---> Z=7f
Test passed for XOR: A=14, B=6c ---> Z=78
Test passed for XOR: A=14, B=6d ---> Z=79
Test passed for XOR: A=14, B=6e ---> Z=7a
Test passed for XOR: A=14, B=6f ---> Z=7b
Test passed for XOR: A=14, B=70 ---> Z=64
Test passed for XOR: A=14, B=71 ---> Z=65
Test passed for XOR: A=14, B=72 ---> Z=66
Test passed for XOR: A=14, B=73 ---> Z=67
Test passed for XOR: A=14, B=74 ---> Z=60
Test passed for XOR: A=14, B=75 ---> Z=61
Test passed for XOR: A=14, B=76 ---> Z=62
Test passed for XOR: A=14, B=77 ---> Z=63
Test passed for XOR: A=14, B=78 ---> Z=6c
Test passed for XOR: A=14, B=79 ---> Z=6d
Test passed for XOR: A=14, B=7a ---> Z=6e
Test passed for XOR: A=14, B=7b ---> Z=6f
Test passed for XOR: A=14, B=7c ---> Z=68
Test passed for XOR: A=14, B=7d ---> Z=69
Test passed for XOR: A=14, B=7e ---> Z=6a
Test passed for XOR: A=14, B=7f ---> Z=6b
Test passed for XOR: A=14, B=80 ---> Z=94
Test passed for XOR: A=14, B=81 ---> Z=95
Test passed for XOR: A=14, B=82 ---> Z=96
Test passed for XOR: A=14, B=83 ---> Z=97
Test passed for XOR: A=14, B=84 ---> Z=90
Test passed for XOR: A=14, B=85 ---> Z=91
Test passed for XOR: A=14, B=86 ---> Z=92
Test passed for XOR: A=14, B=87 ---> Z=93
Test passed for XOR: A=14, B=88 ---> Z=9c
Test passed for XOR: A=14, B=89 ---> Z=9d
Test passed for XOR: A=14, B=8a ---> Z=9e
Test passed for XOR: A=14, B=8b ---> Z=9f
Test passed for XOR: A=14, B=8c ---> Z=98
Test passed for XOR: A=14, B=8d ---> Z=99
Test passed for XOR: A=14, B=8e ---> Z=9a
```

```
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
Test passed for NOT: A=4b ---> Z=b4
```

```
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test passed for LRS: A=ff ---> Z=7f
Test completed
PS C:\Users\soyjo\Documents\Programas_Verilog>
```

```
//ALU_M testbench
`timescale 1ns/1ps
`include "ALU_M.v" //Import the file to which we are making its testbench

module ALU_MAuto_tb ();
integer sel_var, A_var, B_var; //Variable for counting
reg  [7:0] A,B; //8-bit inputs
reg  [2:0] sel; //3-bit select input
wire [7:0] Z; //8-bit output
wire zero,carry,sign; //bitFlags

//Instantiate the module under test
ALU_M uut (A,B,sel,Z,zero,carry,sign);

initial begin

  $dumpfile("ALU_MAuto_tb.vcd"); //Create the file that will give us the waveform of our
output
  $dumpvars(0,ALU_MAuto_tb);    //Record all variable changes in the testbench

for (sel_var = 0; sel_var < 9; sel_var = sel_var + 1) begin
  sel = sel_var[2:0];
 for (A_var = 0; A_var < 256; A_var = A_var + 1) begin
    A = A_var[7:0];
    for (B_var = 0; B_var < 256; B_var = B_var + 1) begin
       B = B_var[7:0];
       #10;
       if (sel_var == 0) begin
          if (Z !== (A + B)) begin
             $display("Test failed for ADD: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A +
B), Z);
          end else begin
             $display("Test passed for ADD: A=%h, B=%h ---> Z=%h", A, B, Z);
          end
       end

       if (sel_var == 1) begin
          if (Z !== (A - B)) begin
             $display("Test failed for SUB: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A -
B), Z);
          end else begin
             $display("Test passed for SUB: A=%h, B=%h ---> Z=%h", A, B, Z);
          end
```

```
        end

        if (sel_var == 2) begin
          if (Z !== (A & B)) begin
            $display("Test failed for AND: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A &
B), Z);
          end else begin
            $display("Test passed for AND: A=%h, B=%h ---> Z=%h", A, B, Z);
          end
        end

        if (sel_var == 3) begin
          if (Z !== (A | B)) begin
            $display("Test failed for OR: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A |
B), Z);
          end else begin
            $display("Test passed for OR: A=%h, B=%h ---> Z=%h", A, B, Z);
          end
        end

        if (sel_var == 4) begin
          if (Z !== (A ^ B)) begin
            $display("Test failed for XOR: A=%h, B=%h ---> Expected Z=%h, got Z=%h", A, B, (A ^
B), Z);
          end else begin
            $display("Test passed for XOR: A=%h, B=%h ---> Z=%h", A, B, Z);
          end
        end

        if (sel_var == 5) begin
          if (Z !== (~A)) begin
            $display("Test failed for NOT: A=%h ---> Expected Z=%h, got Z=%h", A, (~A), Z);
          end else begin
            $display("Test passed for NOT: A=%h ---> Z=%h", A, Z);
          end
        end

        if (sel_var == 6) begin
          if (Z !== ((A << 1))) begin
            $display("Test failed for LLS: A=%h ---> Expected Z=%h, got Z=%h", A, (A << 1), Z);
          end else begin
            $display("Test passed for LLS: A=%h ---> Z=%h", A, Z);
          end
        end
```

```
        if (sel_var == 7) begin
          if (Z !== (A >> 1)) begin
            $display("Test failed for LRS: A=%h ---> Expected Z=%h, got Z=%h", A, (A >> 1), Z);
          end else begin
            $display("Test passed for LRS: A=%h ---> Z=%h", A, Z);
          end
        end

    end
  end
end

  $display("Test completed"); //Show message in console when test is completed

end

endmodule
```